

Replication: Policy Learning with Adaptively Collected Data

Zhan, Ren, Athey, and Zhou (2024), Management Science 70(8)

Weiheng Zhang

2026-09-05

Contents

1	The paper and the question	1
2	The pieces in R	2
2.1	Data-generating process (their <code>one_dim_data</code>)	2
2.2	Floored linear Thompson sampling (their <code>LinTSMModel</code> , <code>draw_thompson</code> , <code>apply_floor</code>)	2
2.3	Leave-future-out outcome models (their <code>ridge_muhat_lfo_pai</code>)	3
2.4	Scores with <code>dr_scores()</code> and a check against the authors' formula	3
3	Synthetic experiment (their Figure 2)	4
4	Weighted evaluation	7
5	Classification data sets (their Figure 3 and Table 2)	7
6	What the workflow teaches	9
7	References	10

1 The paper and the question

Policy learning as in Section 08 assumes independent draws from a fixed assignment rule. Zhan, Ren, Athey, and Zhou (2024) drop that assumption. Their data come from a contextual bandit: at each time t a context X_t arrives, an action $W_t \in \{1, \dots, K\}$ is drawn with probabilities $e_t(X_t, w)$ that depend on everything observed before t , and $Y_t = \mu(X_t, W_t) + \varepsilon_t$ is observed. Only $e_t(X_t, W_t)$, the probability of the action taken, is recorded. Two things go wrong for off-line policy learning. Samples are dependent. And exploration shrinks: arms that looked bad early are rarely pulled later, so the counterfactual outcomes of those arms are estimated from few, heavily weighted observations.

Their answer keeps the doubly robust score and adds one ingredient. With $\hat{\mu}_t(x, w)$ fitted on the data *before* t (leave-future-out),

$$\hat{\Gamma}_t(w) = \hat{\mu}_t(X_t, w) + \frac{\mathbb{1}\{W_t = w\}}{e_t(X_t, w)} (Y_t - \hat{\mu}_t(X_t, w)),$$

and the generalized AIPW value of a policy π with prespecified weights h_t is

$$\hat{Q}(\pi) = \frac{\sum_{t=1}^T h_t \hat{\Gamma}_t(\pi(X_t))}{\sum_{t=1}^T h_t}, \quad \hat{\pi} = \arg \max_{\pi \in \Pi} \hat{Q}(\pi).$$

If the assignment probabilities are bounded below by a known nonincreasing sequence g_t , the weights $h_t = g_t$ minimize the worst-case variance and give the minimax regret rate; otherwise $h_t = 1$ is the recommended

default (their Algorithm 1). In the case study of their Section 4.3.3, $g_t = t^{-\alpha}$ and $h_t = t^{-\beta}$: any $\beta < (\alpha+1)/2$ keeps the rate $T^{(\alpha-1)/2}$, and $\beta = \alpha$ is best; $\beta = 2\alpha$ violates their Assumption 2 and has no guarantee.

The package already has every piece of the causal layer: `dr_scores()` with the recorded propensities and supplied nuisances builds $\hat{\Gamma}_t$; `policy_learn(weights = h_t)` and `policy_value(weights = h_t)` implement the generalized AIPW objective and the adaptively weighted standard error. The bandit that collects the data is written below, following the authors' Python package (`utils/`) line by line.

2 The pieces in R

2.1 Data-generating process (their `one_dim_data`)

Two arms, $X \sim U[-2, 2]^3$, $\mu_1(x) = x_1^2$, $\mu_2(x) = 2 - x_1^2$ (the code's `signal = 1`, `bias = 1`), standard normal noise. The optimal policy is a depth-two tree in x_1 (arm 2 when $|x_1| < 1$) with value $B^2/3 - 1 + 4/(3B) + 1 = 2$ for $B = 2$.

```
one_dim_data <- function(n, p = 3, B = 2, noise_sd = 1, signal = 1, bias = 1) {
  xs <- matrix(runif(n * p, -B, B), n, p, dimnames = list(NULL, paste0("x", seq_len(p))))
  mu <- cbind(xs[, 1]^2 - 1, 1 - xs[, 1]^2) * signal + bias
  list(xs = xs, mu = mu, ys = mu + matrix(rnorm(n * 2, sd = noise_sd), n, 2))
}
value_optimal <- function(B = 2, signal = 1, bias = 1) B^2 / 3 - 1 + 4 / (3 * B) * signal + bias
value_optimal()
#> [1] 2
```

2.2 Floored linear Thompson sampling (their `LinTSModel`, `draw_thompson`, `apply_floor`)

Per arm, a ridge regression of Y on $(1, X)$ with the penalty chosen by leave-one-out cross-validation over the authors' grid, and a Gaussian posterior with covariance $\hat{\sigma}^2(X'X + \lambda I)^{-1}$. For each context in the next batch, 100 posterior draws per arm give the probability that each arm is best; the floor $g_t = t^{-\alpha}/K$ is imposed by raising probabilities below it and shrinking the others so they still sum to one. The first batch of $50K$ observations is assigned uniformly; later batches have 100 observations.

```
ridge_cv <- function(X, y, alphas = c(1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 1)) {
  X1 <- cbind(1, X)
  XtX <- crossprod(X1); Xty <- crossprod(X1, y)
  loo <- sapply(alphas, function(a) {
    Ainv <- solve(XtX + a * diag(ncol(X1)))
    beta <- Ainv %*% Xty
    e <- as.numeric(y - X1 %*% beta)
    h <- rowSums((X1 %*% Ainv) * X1)
    mean((e / (1 - h))^2)
  })
  a <- alphas[which.min(loo)]
  Ainv <- solve(XtX + a * diag(ncol(X1)))
  beta <- as.numeric(Ainv %*% Xty)
  s2 <- mean((y - X1 %*% beta)^2)
  list(beta = beta, V = s2 * Ainv)
}
apply_floor <- function(p, floor) {
  new <- pmax(p, floor)
  slack <- sum(new) - 1
  ind <- new - floor
  if (sum(ind) <= 0) return(rep(1 / length(p), length(p)))
  new - slack / sum(ind) * ind
}
# probability that each arm is best, from `n_mc` posterior draws per arm
ts_probs <- function(xs, posts, n_mc = 100) {
  X1 <- cbind(1, xs)
  K <- length(posts)
  draws <- lapply(posts, function(ps) X1 %*% t(MASS::mvrnorm(n_mc, ps$beta, ps$V))) # K matrices n x n_mc
  best <- matrix(1L, nrow(X1), n_mc); top <- draws[[1]]
```

```

for (k in seq_len(K)[-1]) {
  b <- draws[[k]] > top
  best[b] <- k; top[b] <- draws[[k]][b]
}
sapply(seq_len(K), function(k) rowMeans(best == k))
}
run_lints <- function(xs, ys, floor_start = 1 / ncol(ys), floor_decay = alpha_floor,
  batch = 100, explore = 50 * ncol(ys), n_mc = 100, recorded_T = NULL) {
  n <- nrow(xs); K <- ncol(ys)
  cs <- cumsum(c(explore, rep(batch, ceiling((n - explore) / batch))))
  cs <- cs[cs <= n]; if (max(cs) < n) cs <- c(cs, n)
  ws <- integer(n); yobs <- numeric(n); e_obs <- numeric(n); e_all <- matrix(NA_real_, n, K)
  ws[1:cs[1]] <- (seq_len(cs[1]) - 1) %% K + 1
  yobs[1:cs[1]] <- ys[cbind(1:cs[1], ws[1:cs[1]])]
  e_obs[1:cs[1]] <- 1 / K; e_all[1:cs[1], ] <- 1 / K
  posts <- lapply(1:K, function(k) { i <- which(ws[1:cs[1]] == k); ridge_cv(xs[i, , drop = FALSE], yobs[i]) })
  agents <- list()
  for (b in seq_along(cs)[-1]) {
    f <- cs[b - 1] + 1; l <- cs[b]
    floor <- floor_start / (f - 1)^floor_decay
    pr <- ts_probs(xs[f:l, , drop = FALSE], posts, n_mc)
    pr <- t(apply(pr, 1, apply_floor, floor = floor))
    w <- sapply(seq_len(nrow(pr)), function(i) sample.int(K, 1, prob = pr[i, ]))
    ws[f:l] <- w; yobs[f:l] <- ys[cbind(f:l, w)]
    e_obs[f:l] <- pr[cbind(seq_len(nrow(pr)), w)]; e_all[f:l, ] <- pr
    posts <- lapply(1:K, function(k) { i <- which(ws[f:l] == k); ridge_cv(xs[i, , drop = FALSE], yobs[i]) })
    if (!is.null(recorded_T) && l %in% recorded_T) agents[[as.character(l)]] <- posts
  }
  list(ws = ws, yobs = yobs, e_obs = e_obs, e_all = e_all, batches = cs, agents = agents)
}

```

2.3 Leave-future-out outcome models (their ridge_muhat_lfo_pai)

For each batch and each arm, a ridge regression (penalty 1, as in the authors' initialization of $A = I$) is fitted on all observations before the batch and predicts the batch. No observation's own outcome enters its prediction, and no later observation does either.

```

lfo_ridge <- function(xs, ws, yobs, K, batches, lambda = 1) {
  n <- nrow(xs); mu <- matrix(0, n, K)
  X1 <- cbind(1, xs)
  for (b in seq_along(batches)[-1]) {
    f <- batches[b - 1] + 1; l <- batches[b]
    for (k in 1:K) {
      i <- which(ws[1:(f - 1)] == k)
      if (length(i) == 0) next
      A <- crossprod(X1[i, , drop = FALSE]) + lambda * diag(ncol(X1))
      mu[f:l, k] <- X1[f:l, , drop = FALSE] %*% solve(A, crossprod(X1[i, , drop = FALSE], yobs[i]))
    }
  }
  mu
}

```

2.4 Scores with dr_scores() and a check against the authors' formula

With two arms the authors' score matrix `gammahat` (their `aw_scores`) has columns $\hat{\Gamma}_t(1)$ and $\hat{\Gamma}_t(2)$; the difference is the binary pseudo-outcome of `dr_scores()` with the recorded propensity of arm 2 and the leave-future-out nuisances. The type = "ipw" variant sets $\hat{\mu} = 0$, as the authors' `gammahat_ipw` does.

```

make_bandit_frame <- function(xs, run, mu) {
  d <- as.data.frame(xs)
  d$w <- run$ws - 1L # arm 2 coded as treatment 1
  d$y <- run$yobs
  d$e2 <- ifelse(d$w == 1L, run$e_obs, 1 - run$e_obs) # recorded propensity of arm 2
  d$mu1 <- mu[, 1]; d$mu2 <- mu[, 2]
  d
}

```

```

}
set.seed(1)
demo <- one_dim_data(4000)
run <- run_lints(demo$xs, demo$ys)
mu_lfo <- lfo_ridge(demo$xs, run$ws, run$yobs, 2, run$batches)
frame <- make_bandit_frame(demo$xs, run, mu_lfo)
sc <- dr_scores(frame, y = "y", d = "w", x = c("x1", "x2", "x3"), p_hat = "e2",
  mu0_hat = "mu1", mu1_hat = "mu2", p_clip = c(0, 1))
authors_gamma <- function(frame) {
  bal <- 1 / ifelse(frame$w == 1L, frame$e2, 1 - frame$e2)
  g1 <- frame$mu1 + (frame$w == 0L) * bal * (frame$y - frame$mu1)
  g2 <- frame$mu2 + (frame$w == 1L) * bal * (frame$y - frame$mu2)
  cbind(g1, g2)
}
G <- authors_gamma(frame)
c(max_abs_diff = max(abs(sc$score - (G[, 2] - G[, 1]))),
  share_arm2 = mean(frame$w), min_propensity = min(run$e_obs))
#> max_abs_diff      share_arm2 min_propensity
#> 2.842171e-14      4.425000e-02  8.333333e-03

```

The agent has a misspecified linear model of a quadratic outcome. It learns that arm 1 is better on average ($E[x_1^2] = 4/3$ against $2/3$) and pulls arm 2 with probability near the floor, which is what makes the off-line problem hard: the observations that identify where arm 2 is better are few and carry weights of order $1/g_t$. Figure 1 shows the floor and the weights $h_t = t^{-\beta}$ that the experiments compare.

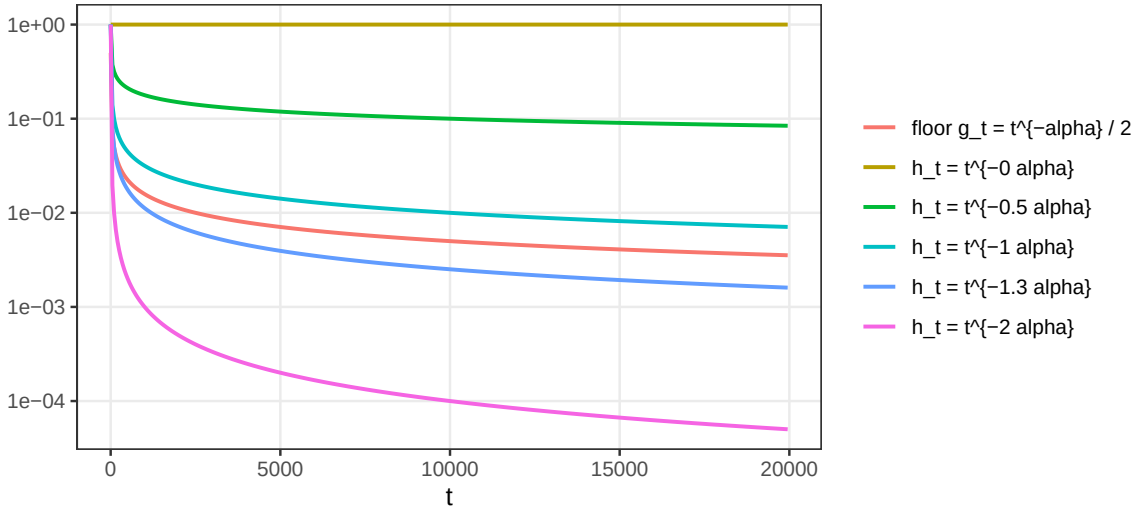


Figure 1: The assignment-probability floor of the data-collection agent and the weighting schemes compared in the experiments, on a log scale. Weights are normalized to one at $t = 1$.

3 Synthetic experiment (their Figure 2)

The authors record the data at $T = 2,000, 4,000, \dots, 20,000$, learn a depth-two tree at each T with weights $h_t \in \{1, t^{-\alpha/2}, t^{-\alpha}, t^{-1.3\alpha}, t^{-2\alpha}\}$, and report the regret of each learned policy and of the agent itself on 100,000 fresh draws, averaged over 1,000 replications. We run 100 replications. The tree search is the package's depth-two search (root splits on a grid of 25 quantiles per variable, children exact); the authors call `policytree`, which gives the same trees up to the grid. IPW scores are learned at $T = 20,000$ only.

```

recorded_T <- seq(2000, 20000, by = 2000)
betas <- c(0, 0.5, 1, 1.3, 2)
beta_label <- c("h_t = 1", "h_t = t^{-0.5a}", "h_t = t^{-a (optimal)", "h_t = t^{-1.3a}", "h_t = t^{-2a}")
policy_regret <- function(pol, test) {
  w <- predict(pol, as.data.frame(test$xs)) + 1
  value_optimal() - mean(test$mu[cbind(seq_len(nrow(test$mu)), w)])
}

```

```

}
agent_regret <- function(posts, test, n_mc = 50, chunk = 10000) {
  n <- nrow(test$xs); val <- 0
  for (s in seq(1, n, by = chunk)) {
    i <- s:min(s + chunk - 1, n)
    pr <- ts_probs(test$xs[i, ], drop = FALSE, posts, n_mc)
    val <- val + sum(rowSums(pr * test$mu[i, ]))
  }
  value_optimal() - val / n
}

one_synthetic_sim <- function(test) {
  dat <- one_dim_data(20000)
  run <- run_lints(dat$xs, dat$ys, recorded_T = recorded_T)
  mu <- lfo_ridge(dat$xs, run$ws, run$yobs, 2, run$batches)
  frame <- make_bandit_frame(dat$xs, run, mu)
  rows <- list()
  for (TT in recorded_T) {
    sc <- dr_scores(frame[1:TT, ], "y", "w", c("x1", "x2", "x3"), p_hat = "e2",
                    mu0_hat = "mu1", mu1_hat = "mu2", p_clip = c(0, 1))
    for (b in betas) {
      h <- seq_len(TT)^(-b * alpha_floor)
      pol <- policy_learn(sc, method = "tree", depth = 2, weights = h, holdout = 0, max_root_splits = 25)
      rows[[length(rows) + 1]] <- tibble(T = TT, beta = b, score = "AIPW", regret = policy_regret(pol, test))
      if (TT == max(recorded_T)) {
        sc_ipw <- dr_scores(frame[1:TT, ], "y", "w", c("x1", "x2", "x3"), p_hat = "e2", type = "ipw", p_clip = c(0, 1))
        pol_ipw <- policy_learn(sc_ipw, method = "tree", depth = 2, weights = h, holdout = 0, max_root_splits = 25)
        rows[[length(rows) + 1]] <- tibble(T = TT, beta = b, score = "IPW", regret = policy_regret(pol_ipw, test))
        # weighted evaluation of the AIPW policy (Section 4 of this note)
        sc2 <- dr_scores(frame[1:TT, ], "y", "w", c("x1", "x2", "x3"), arms = c(0, 1),
                        p_hat = cbind(1 - frame$e2[1:TT], frame$e2[1:TT]), mu_hat = c("mu1", "mu2"), p_clip = c(0, 1))
        truth <- mean(test$mu[cbind(seq_len(nrow(test$mu)), predict(pol, as.data.frame(test$xs)) + 1)])
        for (bb in c(0, 1)) {
          hh <- seq_len(TT)^(-bb * alpha_floor)
          v <- policy_value(sc2, predict(pol, sc2$data), weights = hh)
          rows[[length(rows) + 1]] <- tibble(T = TT, beta = b, score = paste0("value_h", bb), regret = v$estimate - truth,
                                            se = v$std.error, covered = abs(v$estimate - truth) <= 1.96 * v$std.error)
        }
      }
    }
    rows[[length(rows) + 1]] <- tibble(T = TT, beta = NA, score = "TS agent", regret = agent_regret(run$agents[[as.character(TT)]]))
  }
  bind_rows(rows)
}

# Replications run on four cores with their own seeds; the test data are
# drawn once. The root-split grid has 25 points per variable (the exact
# search on all thresholds gives the same trees here and is four times slower).
set.seed(2024)
test_data <- one_dim_data(100000)
n_cores <- max(1L, min(4L, parallel::detectCores() - 1L))
synth <- bind_rows(parallel::mclapply(seq_len(params$n_sims_synth), function(s) {
  set.seed(1000 + s)
  one_synthetic_sim(test_data) %>% mutate(sim = s)
}, mc.cores = n_cores))

```

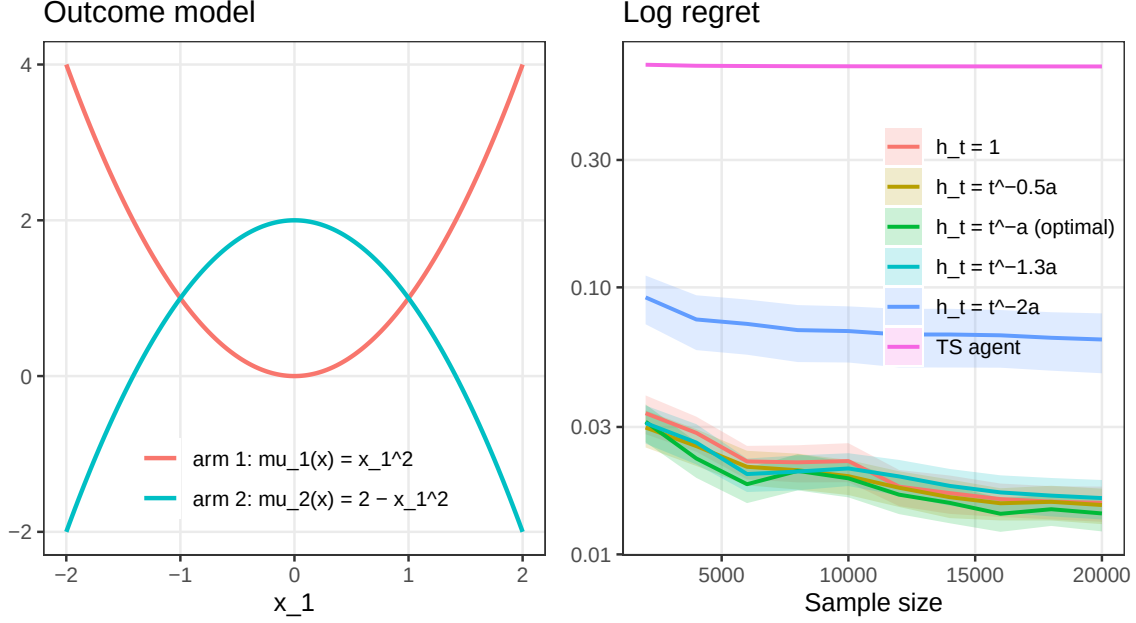


Figure 2: Replication of the paper's Figure 2. Left: the two arm outcome models. Right: out-of-sample regret (log scale) of the depth-two tree learned at each sample size with AIPW scores and weights h_t , and of the Thompson sampling agent's own policy; bands are 95 percent confidence intervals of the mean across replications.

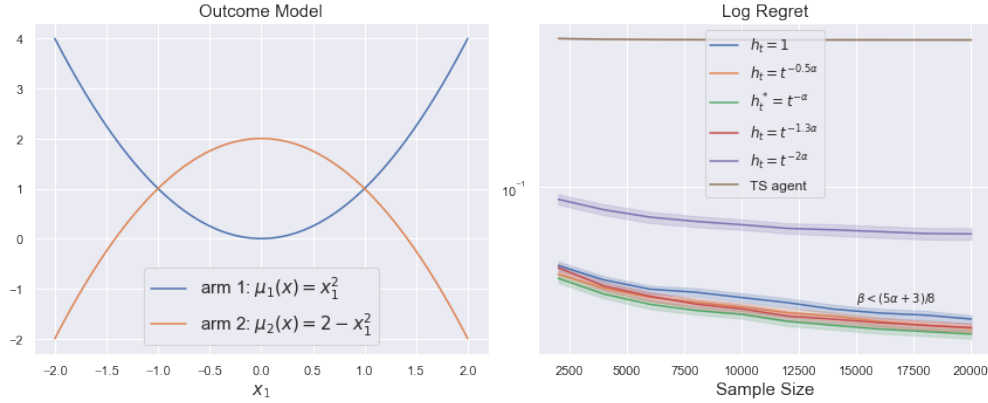


Figure 3: The authors' Figure 2 as embedded in their plotting notebook (1,000 replications).

Table 1: Out-of-sample regret at $T = 20,000$ by weighting scheme and score type, mean (standard error of the mean) over 100 replications. The agent's own regret at $T = 20,000$ is 0.672.

Weights	AIPW scores	IPW scores
$h_t = 1$	0.016 (0.001)	0.023 (0.002)
$h_t = t^{-0.5a}$	0.015 (0.001)	0.020 (0.002)
$h_t = t^{-a}$ (optimal)	0.014 (0.001)	0.019 (0.002)
$h_t = t^{-1.3a}$	0.016 (0.001)	0.021 (0.002)
$h_t = t^{-2a}$	0.064 (0.008)	0.066 (0.009)

Three findings of the paper reproduce.

- The agent’s own policy has a regret of about 0.67 at every sample size: its linear model cannot represent the quadratic outcome, so more data do not help it. Every off-line policy learned from the same data does far better, because the tree class contains the truth and the doubly robust scores do not need the outcome model to be right.
- Among the weightings, $h_t = t^{-2\alpha}$ is worst (regret 0.064 at $T = 20,000$) and decays slowly, as Assumption 2 fails for it. The four schemes with $\beta < (5\alpha + 3)/8$ decay at the same rate and sit close together. The optimal weights $t^{-\alpha}$ are best at $T = 20,000$ (regret 0.014), as in the paper. With 100 replications the ordering among the four is within the error bands, as it nearly is in the authors’ figure with 1,000.
- IPW scores (Table 1) are worse than AIPW scores at every weighting: the outcome model, though misspecified, removes variance from the inverse-propensity term.

4 Weighted evaluation

The paper is about learning, but the same weights apply to evaluation: the generalized AIPW estimator $\hat{Q}(\pi)$ with its adaptive standard error $\sqrt{\sum_t h_t^2 (\hat{\Gamma}_t(\pi(X_t)) - \hat{Q})^2 / \sum_t h_t}$ (their Section 4.1; Hadad et al. 2021). For the policy learned with the optimal weights at $T = 20,000$, we compare the value estimated on the training data with $h_t = 1$ and $h_t = t^{-\alpha}$ against its true value on the test data. `policy_value()` with a two-arm score object (`arms = c(0, 1)`) returns the expected outcome under the policy rather than a gain.

Table 2: Estimated value of the learned policy (optimal weights, $T = 20,000$) on the training data against its true test-set value, across replications.

Weights	Bias	RMSE	Mean SE	95% coverage
$h_t = 1$	0.043	0.109	0.107	0.960
$h_t = t^{-\alpha}$	0.046	0.092	0.091	0.940

Both estimates are slightly optimistic: the bias of about 0.04 is 0.5 standard errors, the winner’s curse of evaluating a policy on the data that selected it, which the paper avoids by evaluating on fresh draws. Coverage of the 95 percent intervals is at the nominal rate for both. The weighted estimator has the smaller standard error here: late observations, where the agent pulls arm 2 with probability near the floor and the inverse-propensity terms are largest, receive the smallest weights.

5 Classification data sets (their Figure 3 and Table 2)

The authors turn 82 OpenML classification data sets into bandits: each class is an arm, the potential outcome of an arm is one if it equals the label and zero otherwise, plus standard normal noise; a floored linear Thompson sampling agent with K arms collects $T = \frac{3}{4} \min(n, 30,000)$ observations, batches of $\min(100, T/10)$ and an initial uniform batch of $K \cdot \text{batch}/2$; the remaining rows are the test set; regret is one minus the test accuracy of the learned depth-two tree. Weights $h_t \in \{1, t^{-\alpha/2}, t^{-\alpha}, t^{-2\alpha}\}$; 150 replications per data set. We use 12 data sets with numeric features and at most twenty of them (the exact tree search scales with the number of features), 20 replications each, and the multi-arm scores of `dr_scores()` for every data set, binary or not.

```
datasets <- tribble(
  ~name, ~id,
  "banknote-authentication", 1462, "wilt", 40983, "mammography", 310, "delta_ailerons", 803,
  "phoneme", 1489, "mozilla4", 1046, "ringnorm", 1496, "twonorm", 1507,
  "page-blocks", 30, "volcanoes-b3", 1533, "artificial-characters", 1459, "pendigits", 32
) %>% slice_head(n = params$n_datasets)
load_openml <- function(id, name) {
  cache <- file.path(data_path, "openml_cache", paste0(name, ".rds"))
  if (file.exists(cache)) return(readRDS(cache))
}
```

```

d <- mlr3oml::odt(id = id)
X <- as.data.frame(d$data[, setdiff(names(d$data), d$target_names), drop = FALSE])
X <- as.matrix(data.frame(lapply(X, function(v) as.numeric(as.character(v)))))
X[!is.finite(X)] <- 0
y <- as.integer(factor(d$data[[d$target_names]]))
out <- list(X = X, y = y, name = name)
saveRDS(out, cache)
out
}

one_class_sim <- function(ds, betas = c(0, 0.5, 1, 2)) {
  n_all <- min(nrow(ds$X), 30000); idx <- sample.int(nrow(ds$X))
  K <- max(ds$y); n <- floor(n_all * 3 / 4)
  train_i <- idx[1:n]; test_i <- idx[-(1:n)]
  xs <- ds$X[train_i, , drop = FALSE]; colnames(xs) <- paste0("x", seq_len(ncol(xs)))
  mu <- diag(K)[ds$y[train_i], , drop = FALSE]
  ys <- mu + matrix(rnorm(n * K), n, K)
  batch <- min(100, n %/% 10)
  run <- run_lints(xs, ys, floor_start = 1 / K, batch = batch, explore = (batch %/% 2) * K)
  mu_lfo <- lfo_ridge(xs, run$ws, run$yobs, K, run$batches)
  frame <- as.data.frame(xs); frame$w <- run$ws; frame$y <- run$yobs
  sc <- dr_scores(frame, "y", "w", colnames(xs), arms = 1:K, p_hat = run$e_all, mu_hat = mu_lfo, p_clip = c(0, 1))
  xs_test <- as.data.frame(ds$X[test_i, , drop = FALSE]); names(xs_test) <- colnames(xs)
  bind_rows(lapply(betas, function(b) {
    h <- seq_len(n)^(-b * alpha_floor)
    pol <- policy_learn(sc, method = "tree", depth = 2, weights = h, holdout = 0, max_root_splits = 30)
    acc <- mean(as.integer(as.character(predict(pol, xs_test))) == ds$y[test_i])
    tibble(dataset = ds$name, n = n, K = K, p = ncol(xs), beta = b, regret = 1 - acc)
  })))
}

class_res <- bind_rows(lapply(seq_len(nrow(datasets)), function(j) {
  ds <- load_openml(datasets$id[j], datasets$name[j])
  bind_rows(parallel::mclapply(seq_len(params$n_sims_class), function(s) {
    set.seed(2000 + 100 * j + s)
    one_class_sim(ds) %>% mutate(sim = s)
  }, mc.cores = n_cores)))
})

```

Table 3: OpenML data sets used, in the paper’s Table 1 categories: features, classes, observations.

Data set	Bandit sample T	Arms K	Features p
artificial-characters	7663	10	7
banknote-authentication	1029	2	4
delta_aileron	5346	2	5
mammography	8387	2	6
mozilla4	11658	2	5
page-blocks	4104	5	10
pendigits	8244	10	16
phoneme	4053	2	5
ringnorm	5550	2	20
twonorm	5550	2	20
volcanoes-b3	7789	5	3
wilt	3629	2	5

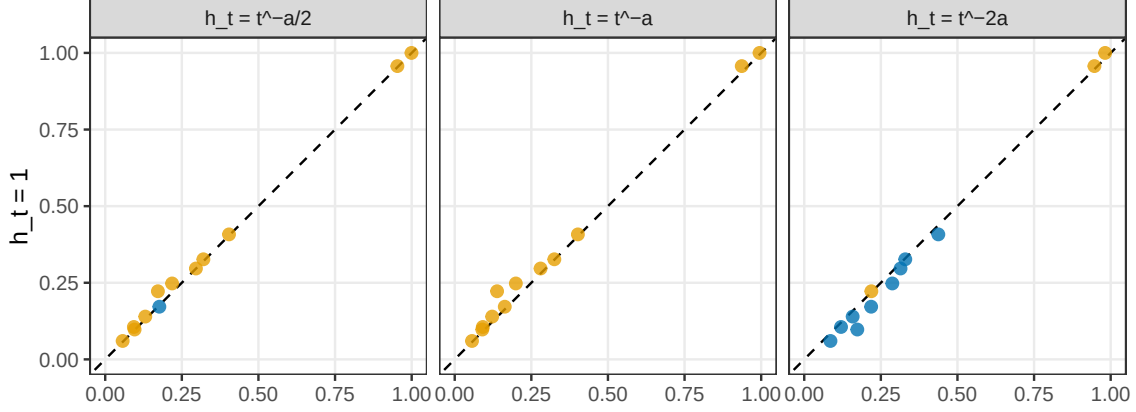


Figure 4: Replication of the paper’s Figure 3. Each point is a data set; coordinates are the regret (one minus test accuracy, averaged over replications) of the tree learned with uniform weights (vertical) and with the weights on the horizontal axis, both normalized by the largest regret in the panel. Orange: the alternative weighting beats uniform weights; blue: it does not.

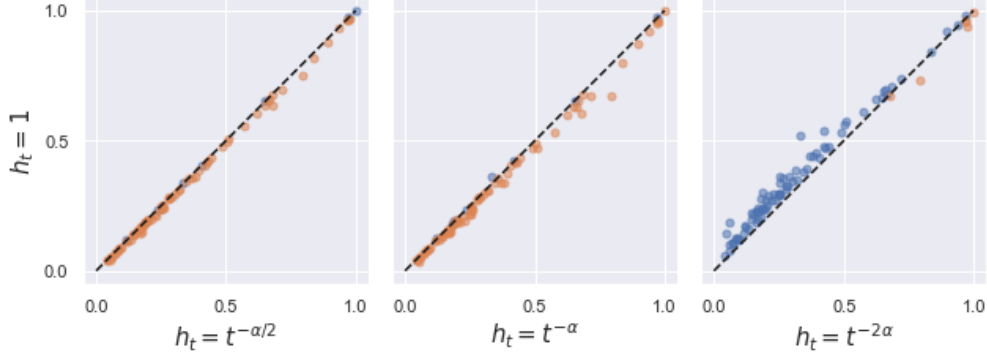


Figure 5: The authors’ Figure 3 as embedded in their plotting notebook (82 data sets, 150 replications each).

Table 4: The paper’s Table 2 on our 12 data sets (the paper: 82 data sets; average regret 0.313, 0.307, 0.301 and worst for $t^{-2\alpha}$; median 0.225, 0.219, 0.215).

Statistic	$h_t = 1$	$h_t = t^{-a/2}$	$h_t = t^{-a}$	$h_t = t^{-2a}$
Average regret over data sets	0.299	0.290	0.282	0.317
Median regret among data sets	0.209	0.176	0.161	0.225
Number of data sets on which the weighting achieves the smallest regret	0	1	10	1

The paper’s pattern appears on our subset: $t^{-\alpha/2}$ and $t^{-\alpha}$ beat uniform weights on 11 and 12 of 12 data sets with mild improvements, and $t^{-2\alpha}$ is worse on 9 of them by a wide margin. The absolute regrets are not comparable to the paper’s because the data sets differ; the ordering of the weightings is what the theory predicts.

6 What the workflow teaches

- **Same score, one new input.** Adaptive data change the propensity from an estimated function to a recorded number and the nuisance fit from cross-fitting to leave-future-out fitting; the AIPW score

itself is unchanged, and `dr_scores()` builds it from the recorded pieces exactly.

- **Weights are the only new decision.** The generalized AIPW objective scales each observation’s score by h_t ; `policy_learn(weights = )` does it for any policy class. With a known floor, $h_t = g_t$; without one, $h_t = 1$ is safe. Over-aggressive down-weighting ($t^{-2\alpha}$) throws away the late, informative observations and shows up immediately.
- **Where the gain over the agent comes from.** The agent must commit to a model to act; the off-line learner only needs a policy class that contains a good rule and scores that are robust to the model. On a misspecified agent the off-line policy is an order of magnitude better, from the agent’s own data.
- **Evaluation inherits the weights.** The same h_t give a valid variance for the value of any fixed policy on adaptively collected data; the cost is a larger standard error than uniform weighting when exploration was in fact adequate.

7 References

- Athey, Susan, and Stefan Wager. 2021. “Policy Learning with Observational Data.” *Econometrica*.
- Hadad, Vitor, David A. Hirshberg, Ruohan Zhan, Stefan Wager, and Susan Athey. 2021. “Confidence Intervals for Policy Evaluation in Adaptive Experiments.” *Proceedings of the National Academy of Sciences*.
- Sverdrup, Erik, Ayush Kanodia, Zhengyuan Zhou, Susan Athey, and Stefan Wager. 2020. “policytree: Policy Learning via Doubly Robust Empirical Welfare Maximization over Trees.” *Journal of Open Source Software*.
- Vanschoren, Joaquin, Jan N. van Rijn, Bernd Bischl, and Luis Torgo. 2013. “OpenML: Networked Science in Machine Learning.” *SIGKDD Explorations*.
- Zhan, Ruohan, Zhimei Ren, Susan Athey, and Zhengyuan Zhou. 2024. “Policy Learning with Adaptively Collected Data.” *Management Science* 70(8), 5270-5297.
- Zhou, Zhengyuan, Susan Athey, and Stefan Wager. 2023. “Offline Multi-Action Policy Learning: Generalization and Optimization.” *Operations Research*.